

Seven Seas A-16

Security Audit

Aug 15, 2024

Version 1.0.0

Table of Contents

- [Introduction](#)
- [Overall Assessment](#)
- [Specification](#)
- [Source Code](#)
- [Issue Descriptions and Recommendations](#)
- [Security Levels Reference](#)
- [Issue Details](#)
- [Disclaimer](#)

Introduction

This document includes the results of the security audit for Seven Seas's smart contract code as found in the section titled 'Source Code'. The security audit was performed by the Macro security team from August 20th to August 22nd 2024 and from September 4th to September 4th 2024.

The purpose of this audit is to review the source code of certain Seven Seas Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro's review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

The following is an aggregation of issues found by the Macro Audit team:

Severity	Count	Acknowledged	Won't Do	Addressed
High	1	-	-	1
Informational	3	-	-	-

Seven Seas was quick to respond to these issues.

Specification

Our understanding of the specification was based on the following sources:

- Discussions with the Seven Seas team.
- Available documentation in the repository.

Trust Model, Assumptions, and Accepted Risks (TMAAR)

- Strategists are trusted to set gas limits appropriately when bridging assets on any of the integrated bridging protocols.

Source Code

The following source code was reviewed during the audit:

- **Repository:** [boring-vault](#)
- **Commit Hash:** `e12c1e320cca1169c9785a8c7d424c38fca8d688`

Specifically, we audited the changes for the following contracts in [PR 88](#).

Source Code	SHA256
<code>src/base/DecodersAndSanitizers/Protocols/KarakDecoderAndSanitizer.sol</code>	<code>08a51bb6b8aab413a6a6726c5104ab577fbdc14aca268afe476e88095f635f1e</code>
<code>src/base/DecodersAndSanitizers/Protocols/PendleRouterDecoderAndSanitizer.sol</code>	<code>d07ccddf90e42d9807ec27b1e52ce018445a7df569f433e08433741267f37793</code>
<code>src/base/DecodersAndSanitizers/Protocols/PumpStakingDecoderAndSanitizer.sol</code>	<code>dacebb25bfec2dfb839dd914c3ffe637854274a4b574d6b512b104d4fccdf87d</code>
<code>src/base/DecodersAndSanitizers/PumpBtcDecoderAndSanitizer.sol</code>	<code>7de7b24297baa896181a39d69ffba2c16b98b7d68e376af0dd54000b380e2cb8</code>
<code>src/base/DecodersAndSanitizers/SymbioticVaultDecoderAndSanitizerFull.sol</code>	<code>4eda5706aa9499c3f21aa6b195630fcbf379d9d21a98dfddc2d065836c579026</code>
<code>src/base/Drones/BoringDrone.sol</code>	<code>24e975b9f3eff98d7da65fe343c32b84dd51b23f20615992e497ac5d0b496a8a</code>
<code>src/base/Drones/DroneLib.sol</code>	<code>7d8ced9634a9ffdcdd1782692708ce34baf4925cc1e61604ba85669c225e7df0</code>

We also audited the changes for the following bridge integration contracts in the [September branch](#).

Source Code	SHA256
src/base/DecodersAndSanitizers/BaseDecoderAndSanitizer.sol	36d82f4d596965288d39d98adfa17d1ba0805e30ed11335cc1dd23646e8c5bd
src/base/DecodersAndSanitizers/BridgingDecoderAndSanitizer.sol	249dce81e0ecbc9f861e961408c1baa813266fa7bd43ebc424f1f430501fe30c
src/base/DecodersAndSanitizers/Protocols/LineaBridgeDecoderAndSanitizer.sol	c55dbf8e886f25a38e0d2b69f050294dbb590cc7fc76e08242a9b0a0768b2ac5
src/base/DecodersAndSanitizers/Protocols/MantleStandardBridgeDecoderAndSanitizer.sol	7999ce0221b646a9ddc28425d301c5e4cbf6324239574d723b05f4759df29cf1
src/base/DecodersAndSanitizers/Protocols/ScrollBridgeDecoderAndSanitizer.sol	8a40a94925316027c6b6f26245de9c36348945c237bf07e60f2b126645cf5a00

Note: This document contains an audit solely of the Solidity contracts listed above. Specifically, the audit pertains only to the contracts themselves, and does not pertain to any other programs or scripts, including deployment scripts.

Issue Descriptions and Recommendations

Click on an issue to jump to it, or scroll down to see them all.

H-1

ETH sent into BoringPuppet is unrecoverable

I-1

Symbiotic contracts are only on testnet and unaudited

I-2

Karak will update to allow delegation

I-3

Strategists are trusted to set a reasonable gas limit when bridging

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:

- How bad things can get (for a vulnerability)
- The significance of an improvement (for a code quality issue)
- The amount of gas saved (for a gas optimization)

2. The high/medium/low **likelihood** of the issue:

- How likely is the issue to occur (for a vulnerability)

3. The overall critical/high/medium/low **severity** of the issue.

This third part – the severity level – is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client must fix the issue, no matter what, because not fixing would mean significant funds/assets WILL be lost.
(H-x) High	We recommend the client must address the issue, no matter what, because not fixing would be very bad, or some funds/assets will be lost, or the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to seriously consider fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albeit not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not be relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Details

H-4

ETH sent into BoringPuppet is unrecoverable

TOPIC

Protocal Design

STATUS

Fixed [↗](#)

Fixed [↗](#)

IMPACT

High

LIKELIHOOD

Medium

`BoringPuppet.sol` is a contract intended to be controlled by the boring vault and used as a alternate address, used for varying purposes while interacting with protocols, like bypassing point reward caps or user deposit caps. It is relatively simple, allowing the boring vault to call any function via it's callback, forwarding value sent into the call:

```
/**
 * @notice This contract in its current state can only be interacted with by the boring vault
 * @notice The real target is extracted from the call data using `extractTarget`
 * @notice The puppet then forwards
 */
fallback() external payable onlyBoringVault {
    // Extract real target from end of calldata
    address target = PuppetLib.extractTargetFromCalldata();

    // Forward call to real target.
    // TODO we could do some verification of `target`, but if it is wrong then
    target.functionCallWithValue(msg.data, msg.value);
}
```

Reference: [BoringPuppet.sol#L36-L48](#)

This fallback function is the only means that the puppet can make calls, and is limited to only sending ETH equal to the amount sent in on that call, since it uses `msg.value`.

The only other function in the contract is a `receive()` function, which handles the case where ETH is sent directly into the contract without any additional calldata. One intent of having this `receive()` function is to allow the puppet to receive ETH as a reward for protocols that offer it, say for staking. If ETH is sent to the contract, however, there is no way to move the ETH out, since its fallback function only forwards the value sent in. This would cause any ETH rewards to be stuck in the contract forever.

Remediations to Consider

Either:

- Add a withdraw function to the puppet that only the boring vault can call
- Specify a value in the additional data sent into the `fallback()` call, like is done with the target address, rather than use `msg.value`.
- Direct any ETH received to the Boring Vault

I-1

Symbiotic contracts are only on testnet and unaudited

TOPIC	IMPACT
Informational	Informational *

Symbiotic vault contracts are currently only deployed on the Holesky testnet, and their audit reports do not cover them or the associated protocol contracts. This could mean there is a audit pending and may change when deployed to mainnet, caution and re-review is advised once that occurs. Notably the vault contracts do not utilize the symbiotic collateral contracts that it’s documentation claims it will use, so it is possible the underlying token deposited into these vaults will be updated to use their associated collateral token.

I-2 Karak will update to allow delegation

TOPIC

Informational

IMPACT

Informational *

Boring vaults intend to interact with Karak contracts by using the [KarakDecoderAndSanitizer](#) which will interact with the [DelegationSupervisor](#) and [VaultSupervisor](#) contracts. However, currently there is no way to actually delegate shares like the DelegateSupervisor's name would suggest. There is a comment in the VaultSupervisor that implies this will be changed in a future update:

```
// Increase shares delegated to operator
// TODO: to be enabled in the next version when delegation is activated
//self.delegationSupervisor.increaseDelegatedShares(staker, vault, shares);
```

Reference: [VaultSupervisor.sol#L274-L276](#)

Karak contracts are upgradeable, which should always be considered with caution, but it is important to be aware that active changes are planned, and so should be watched in case the contracts update its interface or introduces an exploit.

I-3 Strategists are trusted to set a reasonable gas limit when bridging

TOPIC

Trust Assumption

IMPACT

Informational *

When bridging assets, a min gas limit parameter is typically required which is the gas limit used to execute the transaction on destination chain. If the min gas limit

provided is not sufficient to execute the desired transaction, it may not be able to be executed leaving any ETH or tokens attempted to be bridged potentially stuck. It is expected that when strategists set this value it will be set correctly so it will execute as expected.

Alternatively the min gas limit is used by some protocols, like [Scroll](#), to charge a fee to cover the execution cost. If it is set to be much larger than required, additional unnecessary fees will be charged.

Strategists should simulate the desired transactions to determine the gas used, adding a buffer to ensure it will execute.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the Seven Seas team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.